

Executive Summary

We propose an **all-open-source stack** built around modern web and native technologies to deliver a high-performance (120 fps), slick, “liquid glass” trading chart platform. The front-end will use GPU-accelerated graphics (WebGL/Canvas) and advanced UI frameworks for smooth animations (supporting glassmorphism via CSS). Data will come from free and community sources (crypto exchange feeds, limited equity APIs, FX free feeds), aggregated through a custom backend. The app will be a packaged desktop executable (likely using Tauri or Electron) to run natively on Windows.

Key findings:

- **Charting libraries:** Open-source libraries like TradingView’s Lightweight Charts (Apache 2.0), ChartGPU (MIT, WebGPU-based), Apache ECharts (Apache 2.0), and QFChart (Apache 2.0) offer high performance and rich features. Canvas/ WebGL rendering is vastly faster than SVG (Canvas can be ~3–9× faster than SVG for large datasets [61†L314-L322] , and WebGL with GPU handles even more data).
- **Data sources:** Truly free, real-time equity data is scarce. Some limited sources (e.g. Yahoo/Google for delayed quotes, IEX for partial US stocks) exist, but full market data generally requires paid licenses. Crypto markets are freely accessible via exchange WebSockets (Binance, Coinbase, etc.) [44†L83-L90] [47†L171-L178] . FX/free currency rates can be obtained via public APIs (ECB, exchangerate.host) but real-time streaming is typically paid. Futures data similarly has no free consolidated feed.
- **Desktop frameworks:** Electron (JavaScript/HTML) is mature but heavy (even a “Hello World” Electron app is ~150 MB [35†L87-L96]). Tauri (Rust + system WebView) is far lighter (<10 MB bundle [37†L146-L154]) and faster to start, though requiring Rust work. .NET MAUI (C#/XAML) yields native apps (80–150 MB) [37†L146-L154] . Qt (C++/QML) is highly performant (native GPU support, small footprint) but has licensing complexity (LGPL or commercial) [42†L1-L4] . A pragmatic choice is **Tauri with a modern JS UI framework** (e.g. React+WebGL), or native Qt/QML for ultimate speed/efficiency.
- **Backend & architecture:** A microservice-style backend will ingest data from APIs/WebSockets, normalize and cache it (in Redis or a time-series DB), then push updates to the frontend via WebSockets. The design must handle persistent connections (WebSockets are stateful, requiring sticky sessions or broker-based pub/sub [53†L129-L137] [53†L183-L192]) and scale horizontally. A time-series database (QuestDB, TimescaleDB or InfluxDB) will store history.
- **Performance/UX:** Use GPU rendering (WebGL or hardware-accelerated Canvas) to achieve 120 fps. Offload heavy work to Web Workers or WASM for parallelism. Optimize rendering (batch drawing, limiting DOM work) and use high-DPI assets for sharp UI. The “glassmorphism” look can be implemented via CSS `backdrop-filter: blur()` (supported in modern webviews) or equivalent blur/shadow effects in QML.
- **Other considerations:** Security (TLS for all feeds, sandboxing user scripts), open licensing (use permissive MIT/Apache licenses [62†L484-L492]), and legal caution with data (many free feeds are technically unlicensed – avoid liability by clearly disclosing data sources [24†L64-L72]).

Monetization might include paid premium indicators or institutional data while keeping core features free.

The report below analyzes each area in detail, with tables comparing alternatives and diagrams of the proposed system. A sample project roadmap (with sprints, roles, milestones) is provided.

1. Requirements and Design Goals

- **Smooth, high-fidelity UI:** Target 120 fps rendering for zooming, panning, and animations. Use GPU (WebGL) where possible to achieve fluid “liquid” and glassmorphic effects.
- **Free/Open Data:** Use only freely available market data; be prepared for limits (rate caps, delays). Plan to support at least global crypto, major equities, FX, and futures if possible.
- **Full feature set:** Candlestick charts with drawing tools, indicators, overlays, multiple timeframes, real-time updates, multi-pane layouts.
- **Platform:** Windows desktop (single `.exe`) with auto-update. Potential cross-platform design for other OS in future.
- **Architecture:** Modular (frontend, backend, DB), horizontally scalable, and extensible (plugin scripting).
- **Licensing:** All software components (own code and dependencies) must be open-source, permissively licensed. Data licensing issues must be carefully handled (clearly state delays or use of licensed feeds [24†L64-L72]).
- **Developer & testing:** Use modern CI/CD (GitHub Actions/GitLab CI), unit/UI tests. Developer tooling and contributor workflows (linting, PR process) to ensure code quality.

These shape our technology choices.

2. Technology Stack Overview

2.1 Frontend Rendering Technology

- **Canvas vs SVG vs WebGL:** For chart rendering, Canvas and WebGL vastly outperform SVG for large data. As one study notes, *“Canvas rendering can deliver 3–9× faster frame rates than SVG for large datasets, while WebGL with GPU acceleration handles the most demanding visualization workloads”* [61†L314-L322]. We will use GPU-accelerated Canvas/WebGL for drawing plots. SVG might be used only for simple overlays (axes, labels) when convenient (under ~10,000 points [61†L314-L322]).
- **WebGL/Canvas libraries:** We will base rendering on a high-performance JS/TS library (see Section 3). Optionally, we could use WebGPU-based solutions (ChartGPU) if targeting cutting-edge browsers, but WebGL has broad support.
- **UI Frameworks:** A reactive UI framework (React, Vue, or Svelte) can manage components (menus, dialogs, state). Combined with a rendering canvas (or WebGL layer), it provides structure. For glassmorphism, CSS can be used (backdrop-filter, semi-transparent layers) to achieve “frosted glass” UI elements. Libraries like **Tailwind CSS** or custom styles will help implement the translucent look.
- **Language:** JavaScript or TypeScript. TypeScript is recommended for type safety across a large codebase. If using Tauri, Rust will power the backend process, but frontend is still HTML/JS.
- **Packaging:** Likely an Electron or Tauri app bundling Chromium (Electron) or a native WebView (Tauri) to render the HTML/CSS/JS interface. Both support auto-update frameworks.

2.2 Backend Services

- **Ingestion & Normalization:** A backend service (Node.js/Express or Python/FastAPI or Rust) will fetch and aggregate data from APIs/WebSockets. It will normalize symbols/timeframes (e.g. convert exchange-specific formats to a common schema) and handle subscription logic.
- **Caching:** A fast in-memory cache (Redis) will store recent data, reducing API call rates and serving WebSocket pushes.
- **WebSocket Server:** This service (e.g. using Socket.IO or native WebSocket support) pushes updates to the front-end app. It must handle persistent connections, auto-reconnect, and distribution of data by instrument. As discussed below, horizontal scaling will require a message broker (Redis Pub/Sub or Kafka) to fan out data across multiple WebSocket server instances.
- **Database:** A time-series database will store historical OHLCV and indicator data. Possible choices are InfluxDB, TimescaleDB (PostgreSQL extension), or QuestDB. All are open-source: InfluxDB (MIT/Apache 2.0 since v3.0) and QuestDB (Apache 2.0) offer high ingestion rates [58+L242-L250] . TimescaleDB (PostgreSQL, open source) is robust and SQL-compatible. These allow efficient range queries and downsampling for long-term charts.
- **API Gateway (optional):** If planning additional REST APIs or mobile clients, an API gateway can route requests and handle SSL.

2.3 Plugin/Scripting Environment

- **Language Choices:** JavaScript (the same environment as the UI) is easiest for plugin scripting (no extra language runtime needed). Alternatively, embedding a Lua or Python interpreter can allow strategy scripts. Python is popular for finance, but sandboxing Python is harder. WebAssembly (WASM) is an attractive secure option: compile scripts to WASM (e.g. via AssemblyScript or Rust) and run them in a sandbox.
- **Sandboxing:** User scripts must be sandboxed to prevent system abuse. For JS, a technique like Chrome's `Function` with careful restrictions or SES (Secure ECMAScript) can isolate plugins. For Python, use a restricted execution environment (though truly sandboxing Python is non-trivial). WebAssembly modules naturally run in a sandboxed VM.
- **Extensibility:** Expose a controlled API for indicators: for example, allow scripts to query price history and output values at each timestamp. Ensure plugin execution is time-bounded (e.g. limit loops) to avoid freezing the UI.

2.4 CI/CD and Tooling

- **Version Control:** Git with a hosted repo (GitHub/GitLab). Pull-request workflow, code reviews.
- **Linting/Formatting:** ESLint/Prettier (for JS/TS), clang-format or similar for Rust/C++, style guides enforcement.
- **Testing:** Unit tests (Jest for JS), integration tests for data pipelines, end-to-end tests (Cypress or Playwright) for the UI. Automated benchmarks might verify frame rates or rendering speed for large datasets.
- **CI Pipeline:** On each commit, run lint, tests, and build. For Electron/Tauri apps, automate packaging and possibly run a smoke test on the installer.
- **Issue Tracking & Project Management:** Use GitHub Issues/Projects or similar for backlog, sprints, and milestones. Clear CONTRIBUTING guidelines for contributors.

2.5 Recommended Stack Summary

Layer	Options	Notes / Citations
Chart Engine	TradingView Lightweight (Apache 2.0) Chart.js (MIT) Apache ECharts (Apache 2.0) ChartGPU (MIT) D3.js (BSD)* QFChart (Apache 2.0)	Lightweight Charts (35 KB, high performance) [4†L275-L283] [11†L375-L381] ; ECharts supports “10 million data in realtime” [60†L72-L75] ; ChartGPU supports 35M points at 72 FPS [5†L359-L362] .
UI Framework	React/Vue/Svelte + WebGL/Canvas TailwindCSS / Custom CSS	Standard web UI frameworks for components; CSS <code>backdrop-filter</code> for glassmorphism.
Desktop App	Electron Tauri .NET MAUI Qt/QML Avalonia (C#)	Tauri yields ~<10 MB bundles vs Electron ~150 MB [35†L87-L96] [37†L146-L154] . .NET MAUI apps ~80–150 MB [37†L146-L154] . Qt is native+C++, small & fast but LGPL/commercial licensing [42†L1-L4] .
Data Sources	Crypto: Binance, Coinbase, Kraken (WebSocket) Stocks: Yahoo Finance (historical), Alpha Vantage (5 req/min) [24†L55-L63] , IEX Cloud (no longer free) [49†L55-L63] FX: ECB (daily fixes), ExchangeRatesAPI (hourly free) [51†L1-L4] Futures: none truly free (broker feeds or SIP required)	Binance & Coinbase provide free high-frequency WebSocket feeds [44†L83-L90] [47†L171-L178] . Alpha Vantage covers equities/FX/crypto [51†L1-L4] but free-tier is 25 calls/day [50†L1-L4] .
Backend Stack	Node.js/Express or Python/FastAPI or Rust (Tauri sidecar) Redis (cache/pubsub) TimescaleDB/QuestDB/InfluxDB Socket.IO or native WS	Use pub/sub (Redis/Kafka) to scale WebSocket distribution [53†L183-L192] . TSDBs optimized for time-series (e.g. QuestDB excels at high-ingest financial data [58†L242-L250]).
Scripting	JavaScript (V8 / SES sandbox) WebAssembly (Rust/AssemblyScript plugins)	WASM is secure and fast; JS is flexible and native to platform. Python (via embedded interpreter) possible but sandboxing harder.

(Note: D3 is under a BSD-like license and is extremely flexible but requires more code for basic charts.)

3. Charting Libraries Comparison

Below are representative open-source charting libraries relevant to a TradingView-like app:

Library	License	Chart Types	Performance Notes	Key Features
TradingView Lightweight	Apache 2.0	Candlestick, Bars, Line, Area, Baseline, Heikin-Ashi, Renko, Kagi, etc [4†L333-L341]	Small footprint (≈ 35 KB) [4†L275-L283] ; optimized for financial charts.	110+ drawing tools, 100+ built-in indicators [4†L333-L341] .
Chart.js	MIT	Line, Bar, Pie, Doughnut, Radar, Bubble, Scatter, Area, etc.	Canvas-based; medium performance; can decimate large data.	Simple API; plugin ecosystem; tree-shakable v4; supported by major frameworks [61†L355-L363] .
Apache ECharts	Apache 2.0	20+ types: Candlestick, K-Line, Line, Bar, Pie, Scatter, Map, Funnel, etc.	Canvas/SVG; built-in progressive rendering can display “10 million data in realtime” [60†L72-L75] .	Rich config; responsive; dataset transforms (filter, clustering) [60†L72-L75] ; themeable.
D3.js	BSD-like	Completely custom (fundamental toolkit).	SVG by default; can be slow if many elements (SVG DOM overhead) [61†L449-L458] ; can use Canvas/WebGL via extensions.	Ultimate flexibility for custom visualizations; force graphs, maps, etc. Large community.
ChartGPU	MIT	Line, Area, Bar, Scatter, Pie, Candlestick, etc.	WebGPU-based; very high throughput (35M points at ~ 72 FPS [5†L359-L362]).	TypeScript library; optimized for “lots of data” [5†L329-L337] .
QFChart (QuantForge)	Apache 2.0	Candlestick, Bars, Multi-pane, Overlays, etc.	Built on Apache ECharts; optimized for finance.	High-performance time-series rendering; real-time updates; TypeScript API [15†L99-L107] [15†L121-L129] .
Others (non-free)	–	Highcharts, SciChart, AmCharts, etc.	Not open/free (skip for core).	(Mentioned for context; these offer similar features at cost.)

Each library can be extended via plugins or custom code. For example, TradingView's library supports custom **plugins and indicators** [62†L484-L492] . Our platform should allow user scripts (see Section 7) to add indicators on top of base charts.

4. Real-Time Data Sources

Since **all market data must be free**, we rely on the following:

- **Cryptocurrency:** Exchanges like Binance, Coinbase, Kraken provide **free WebSocket APIs** for live tick and order-book data. For instance, Binance Futures offers public streams at `wss://fstream.binance.com/public` (candles, trades) with rate limits (10 messages/sec per connection) [44†L83-L90] [44†L119-L124] . Coinbase's advanced trade feed is publicly accessible via `wss://advanced-trade-ws.coinbase.com` [47†L171-L178] . These give sub-second updates for all major coins.
- **Global Equities:** Truly free real-time equity data is mostly unavailable due to exchange fees [24†L64-L72] . Some limited sources:
 - *Yahoo Finance:* Unofficial, delayed (~15–20 min) data for stocks and indices. Good for history but not true live.
 - *Alpha Vantage:* Free tier (25 calls/day [50†L1-L4]) covers many stocks, ETFs, FX rates, and crypto [51†L1-L4] . Real-time use requires a paid plan beyond the free limit.
 - *IEX Cloud:* Historically had generous free tier, but as of mid-2024 was shut down [49†L55-L63] . Alpaca brokerage API uses partial IEX data (only ~4% of US liquidity) [20†L293-L301] – not sufficient for full charts.
 - *Local brokerage APIs:* Paper (demo) accounts like Alpaca or Interactive Brokers can stream US stocks, but these are tied to brokerage accounts and still use limited feeds.
 - *FMP / Marketstack / Finnhub:* There are some free or freemium stock data APIs (e.g. FinancialModelingPrep, Finnhub) with modest rate limits; they can supply end-of-day or 5-min data for many tickers. However, delays and caps apply.
 - *Licensing Caution:* Any attempt to use a “free” real-time feed must consider legality: *“Realtime and 15-minute delayed US market data is regulated by the stock exchanges... if you find other data sources that are 'free', ensure they are properly licensed”* [24†L64-L72] .
- **FX Rates:** Free FX (currency) data typically comes from central banks or aggregators:
 - *ECB:* Provides daily exchange rates (not streaming).
 - *ExchangeRate-API / exchangeratesapi.io:* Offer free hourly updates for many currencies.
 - *FIXer.io / OXR:* Paid plans, but limited free plans for basic EUR-based rates. Real-time forex usually requires brokerage or paid feed. We can update FX rates via periodic API pulls.
- **Futures:** Market data for futures (e.g. CME, ICE) is almost never free. A few possibilities:
 - *Broker APIs:* e.g. Interactive Brokers (TWS API) can stream futures quotes for account holders, but again it's country/exchange specific.

- *Aggregators*: Some limited delayed futures data may be available from Yahoo or Google (CME E-mini indices). In practice, full futures support likely requires pay – we may only support a subset (e.g. crypto futures from exchanges, or use indices as proxies).

Data Reliability & Limits: Free APIs often have rate limits (e.g. Binance: 10 WS msgs/sec; Alpha Vantage: 25 REST calls/day [\[50†L1-L4\]](#) ; Binance WS disconnections after 24h [\[44†L119-L124\]](#)). We must handle reconnects, data gaps, and display delays where needed.

<div style="page-break-after: always;"></div>

Asset Class	Example Sources	Coverage	Latency / Limits
Crypto	Binance, Coinbase, Kraken (WS)	Global coins/tokens	Real-time (~ms); free (rate-limited).
Stocks	Yahoo Finance (scrape)	~NYSE/NASDAQ (delayed)	~15–20 min delay; unofficial.
	Alpha Vantage (REST) [51†L1-L4]	Global stocks, FX, crypto	Real-time (with API key); 25 calls/day free [50†L1-L4] .
	Finnhub, IEX Cloud (defunct)	US stocks (partial)	IEX Cloud ended; Finnhub has freemium tier.
Forex	ECB (daily), exchangerate.host	Major 170+ pairs	Daily/hourly updates (no WS).
Futures	Brokers (IBKR), Yahoo Indices	US/EU futures indices	Delayed or via account; limited free.

Table: Free/low-cost data sources. No fully free low-latency stock/futures feed exists [\[24†L64-L72\]](#) .

5. Desktop Application Frameworks

To package as a native Windows executable while using web technologies, we compare:

Framework	Language/Engine	Bundle Size	Performance	Update/Packaging	Notes
Electron	Chromium + Node.js	~150 MB for even simple app [35†L87-L96]	Good (uses Chrome V8/ WebGL); high memory footprint	Electron-builder (NSIS/ Squirrel); autoUpdater module (Squirrel) [63†L0-L3]	Mature ecosystem; many plugins (auto-update, menu, notifications); cons: large bundle, high RAM.

Framework	Language/ Engine	Bundle Size	Performance	Update/ Packaging	Notes
Tauri	System WebView2 / WebKit + Rust	<10 MB 【37†L146- L154】	Very fast (Rust core + native WebView) 【37†L146- L154】	Tauri bundler; Updater crate for auto- update	Tiny footprint; uses OS browser engine; secure (Rust); steeper learning curve.
.NET MAUI	C#/.NET native UI	~80–150 MB 【37†L146- L154】	High (native compiled .NET) 【37†L146- L154】	MSIX/AppX or installers; built- in auto-update possible	Windows- focused (also iOS/Android); rich .NET libraries; heavier than Tauri for simple apps.
Qt (QML)	C++ / Qt/ QML	~tens of MB (when static)	Very high (native GPU acceleration)	Qt installer framework; QtUpdater (third-party)	Excellent 2D/3D/ animation performance; LGPL or commercial licensing required 【42†L1- L4】 .
Others	Avalonia (C#), Flutter Desktop (Dart), etc.	Varies	Good	Platform tools	Cross-platform UI toolkits (Avalonia is .NET open-source, Flutter has Linux/Win support).

- Electron apps include a full Chromium; even “Hello World” is ≈ 150 MB 【35†L87-L96】 . Tauri apps are an order of magnitude smaller by using the OS webview 【37†L146-L154】 . For ultimate speed and control, Qt/QML (C++) is very efficient and can easily exceed 120 fps on modern GPUs, but its licensing is more restrictive (free for open-source; commercial license for closed-source use) 【42†L1-L4】 .
- **Auto-Update:** Electron’s `autoUpdater` (Squirrel) works on Windows; Tauri’s build tools include an auto-update mechanism. Packaging tools (electron-builder, Tauri bundler) can create Windows installers (.exe/.msi).
- **UI Implementation:** If using Electron/Tauri, HTML/CSS+WebGL code remains portable. For Qt, the UI would be written in QML (with JavaScript) or C++. QML easily supports fluid animations and blur effects for glassy UI.
- **Recommendation:** For rapid development with web-based charts, **Tauri + React** is compelling (fast, small). Qt/QML is a strong alternative for pure-native performance.

6. Backend Architecture

The system architecture consists of several components (see diagram below):

```
graph LR
    subgraph Market_Data_Sources
        BinanceWS((Binance WS))
        CoinbaseWS((Coinbase WS))
        StockAPI[(Stock APIs<br/>(AlphaVantage/etc))]
        FXAPI[(FX APIs)]
    end
    subgraph Ingestion_Services
        DataIngest[Ingestion Service]
        MessageBroker((Redis Pub/Sub<br/>or Kafka))
        Normalizer[Data Normalizer]
        Cache[(Redis Cache)]
    end
    subgraph Backend_System
        TimeseriesDB[(Time-Series DB)]
        WebSocketServer[WebSocket Server]
    end
    subgraph Desktop_App
        UI[GUI (WebGL/Canvas)]
    end

    BinanceWS -->|stream| DataIngest
    CoinbaseWS -->|stream| DataIngest
    StockAPI -->|REST pulls| DataIngest
    FXAPI -->|REST pulls| DataIngest

    DataIngest --> Normalizer
    Normalizer --> Cache
    Normalizer --> MessageBroker
    Cache --> TimeseriesDB
    MessageBroker --> WebSocketServer
    TimeseriesDB --> WebSocketServer
    WebSocketServer --> UI
```

Explanation of components:

- **Market Data Sources:** Crypto exchange WebSockets (Binance, Coinbase, etc.) push high-frequency ticks. Equity and FX data are polled via REST (Alpha Vantage, ECB, etc.).
- **Ingestion Service:** Consolidates incoming feeds. Written in a backend language (Node.js, Python, or Rust). Performs sanity checks and writes raw data.

- **Normalizer:** Harmonizes symbols/timestamps (e.g., combine different exchange formats, align timezones) and computes derived values (e.g. candle aggregation if needed). Feeds normalized updates to a cache and message broker.
- **Cache (Redis):** Stores recent price data and chart data for quick retrieval. Also used for pub/sub to propagate updates.
- **Message Broker (Redis Pub/Sub or Kafka):** Distributes normalized updates to downstream services, enabling horizontal scaling of the WebSocket server. This decouples ingestion from client distribution.
- **Time-Series DB:** Stores historical data. Can be TimescaleDB (Postgres), QuestDB, InfluxDB, etc. Used for loading historical charts and for any analytics. DataIngest writes to this DB in batches or via CDC.
- **WebSocket Server:** Pushes real-time updates to the desktop app's front-end. Handles many persistent connections; likely needs clustering. The server listens on channels (via Redis Pub/Sub) to broadcast new ticks/aggregates.
- **Desktop App (UI):** Connects to the WebSocket server for live data. Renders charts in real time and handles user interactions.

Scalability and Fault Tolerance: Using Redis/Kafka for pub/sub allows adding more instances of WebSocket servers as user count grows. Because WebSockets are **stateful** and sticky, a load balancer with session affinity (or token-based routing) may be required [53†L183-L192] . Alternatively, client libraries like Socket.IO can distribute clients among available servers if synced via Redis.

7. Storage and Database Choices

Market data is **time-series** by nature. Good DB choices include:

- **TimescaleDB:** PostgreSQL extension for time-series. Open-source (Apache 2.0 core). It offers full SQL, ACID, and mature tooling. Good for aggregations and joins with relational data. (E.g., use for storing metadata and also time-series).
- **QuestDB:** Purpose-built time-series DB (Apache 2.0). Extremely high ingest and query performance for financial data. Benchmarks show it can handle enormous write rates and queries much faster than others (e.g. *"6–13× faster ingestion than TimescaleDB, 43–418× faster for complex queries"* [58†L242-L250]). Supports nanosecond timestamps and SQL queries.
- **InfluxDB 3 (OSS):** Open-source core (MIT/Apache 2.0). Good for high-cardinality data with its Arrow/Parquet storage [55†L89-L98] [58†L242-L250] . Modern InfluxDB 3.0 has a Rust engine, high ingest throughput, but still evolving.
- **Other TSDBs:** OpenTSDB (HBase-backed), ClickHouse (not TSDB but used in some analytics), etc. We focus on true TSDBs due to high write volume and time-based queries.

Trade-offs: QuestDB offers best raw performance (ideal for large tick streams). TimescaleDB offers ease of use and SQL compatibility. InfluxDB has broad community support. Since all have free/community editions, cost is mainly hosting resources.

8. Plugin/Indicator Scripting

To allow user-defined indicators and strategies:

- **Language:** JavaScript/TypeScript is natural (same runtime as UI). We can expose price data to user scripts as arrays and let them compute indicators. Alternatively, allow WebAssembly plugins (Rust or AssemblyScript compiled to WASM), which are fast and sandboxed.
- **Sandboxing:** Execute scripts in an isolated environment. For JavaScript, using `iframe` sandboxes or SES (Secure ECMAScript) can restrict access to only a provided API (no `eval` of arbitrary code, no network/file access). WebAssembly modules run in a sandbox by default.
- **API Exposure:** Provide a limited API (e.g. `getHistory(symbol, timeframe)`, `plotLine(data)`). Prevent file/OS access. Time-limit execution loops to avoid hangs.
- **Security:** All user scripts run locally in the client app, so risk is limited to the user's own machine. If scripts run on the backend, they must be sandboxed (e.g. via a separate unprivileged container or WebAssembly sandbox with resource quotas).
- **Examples:** Many charting apps allow custom indicators via JS or a domain-specific language. We might even support PineScript-like syntax compiled to JS.

9. Performance Optimization

Rendering: Use WebGL shaders or Canvas batch draws. For 120 fps, minimize per-frame work:

- **GPU Acceleration:** Draw candlesticks/bar series as textures or instanced meshes. Use WebGL for heavy series; fallback to Canvas for simple charts.
- **Data Handling:** Pre-compute decimated data for lower zoom levels. On each frame, only update the parts that change.
- **Multithreading:** Use Web Workers to compute indicators or process new data without blocking the UI thread.
- **Memory:** Reuse objects/arrays (avoid GC pressure). Limit history loaded in memory (page in/out).
- **CSS Animations:** Use `will-change` and transform GPU layers for UI animations (menus, transitions). CSS `backdrop-filter` blur for glass effects leverages GPU (supported in Chromium/Tauri webviews).
- **Benchmarking:** Include performance tests in CI (e.g., load 1M points and measure frame rates) to catch regressions.

Citations: The advantage of GPU over Canvas/SVG is known [\[61†L314-L322\]](#) , and specialized libs like ChartGPU have achieved very high FPS even with massive point counts [\[5†L359-L362\]](#) .

10. Security and Privacy

- **Data Security:** Use HTTPS/TLS for all data sources and WS connections. Even “public” feeds should be fetched over secure channels to avoid MITM issues.
- **API Keys:** If using any free-tier keys (Alpha Vantage, etc.), store them securely in the backend; do not expose in the client.
- **Sandboxing:** (As above) User-supplied scripts or plugins must not break out of the sandbox. Only expose minimal APIs.

- **Privacy:** If the app collects any telemetry (e.g. crash reports), ensure user consent and anonymization.
- **Dependency Management:** Regularly audit open-source dependencies for vulnerabilities (use tools like `npm audit`). Pin or lock versions.
- **Licensing:** Ensure any bundled library honors its license. E.g., TradingView's library requires attribution (see its Apache license) [\[62†L484-L492\]](#) .

11. Licensing and Legal Risks

- **Software License:** We will release our code under a permissive license (MIT or Apache 2.0) to encourage use and contribution. All used libraries should also be MIT/Apache if possible. Avoid GPL if we want proprietary integration in the future.
- **Data License:** As noted, live financial data is often restricted. We must clearly state which feeds are used, and highlight that free equity data (if any) is delayed or limited. For example, *“This software relies on Yahoo Finance and free APIs which provide delayed data; real-time trading data from exchanges requires a paid subscription”*. The platform should disclaim that it is not providing regulated quotes.
- **Regulatory:** If offering any trading or investment advice features, be aware of financial regulations (KYC, securities laws) – likely out of scope for a charting tool.
- **Patents/IP:** Charting algorithms (e.g. technical indicators) are generally not patented, but some companies have patented certain UI features (though unlikely to affect us). Stick to standard algorithms.

12. Monetization Strategy

Keeping the core charting tool free (as requested) suggests these optional revenue models:

- **Support/Consulting:** Offer paid enterprise support, custom development, or hosting.
- **Premium Data Bundles:** Offer paid plans for advanced data sources (e.g. real-time exchange data). The app could integrate with a subscription service (like a partnership with a data provider).
- **Add-on Marketplace:** Charge for proprietary indicators, themes, or integration plugins.
- **Donations/Sponsorship:** Since open-source, accept donations (GitHub Sponsors) or display subtle sponsorship (e.g. a “Powered by OpenCharts” badge).
- **White-labeling:** Offer the platform as a branded solution to firms (with a license fee).

All monetization must not lock basic chart functionality behind paywalls, to keep the core truly free.

13. Testing and CI/CD

- **Unit Tests:** Write automated tests for data parsing, indicator calculations, normalization logic. For UI, use a testing framework like Jest (for React) or Vue Test Utils to test components.
- **Integration Tests:** Simulate data feeds to ensure charts update correctly. End-to-end tests (Cypress/Playwright) can automate user flows: e.g. load a chart, zoom/pan, check for regressions.
- **Performance Tests:** Include automated performance benchmarks (e.g. measure FPS under fixed load) to guard against slowdowns.
- **Continuous Integration:** On each commit/PR, run linting, unit tests, and build packaging. For major branches, build installers (using electron-builder or Tauri).

- **Release Process:** Tag versions (e.g. semantic versioning). Use GitHub Releases or similar to publish installers.

14. Deployment and Installer Creation

- **Electron:** Use **electron-builder** to create a Windows `.exe` installer (via NSIS or Squirrel). Configure code-signing for security. Use the built-in `autoUpdater` (Squirrel) for updates [\[63†L0-L3\]](#) .
- **Tauri:** Tauri has its own bundler to produce an `.msi` or `.exe` . It can integrate with Sparkle or other update services on macOS/Windows. Auto-updates can be enabled via the Tauri Updater.
- **Installer UX:** Provide a standard installer wizard. Ensure the app auto-checks for updates on launch (with user control).
- **Packaging:** All assets (charts, libs) are local; however, we might consider packaging any web assets (HTML/CSS) inside the app, so it can run offline (except data feeds).
- **Continuous Deployment:** Optionally, set up a CI pipeline to automatically build and sign the installer when a release is tagged.

15. Developer Tooling and Contributor Workflow

- **Monorepo vs Multiple Repos:** A single repo can hold frontend, backend, and shared config (e.g. a monorepo with subdirectories, managed by tools like Nx or Lerna). This simplifies versioning. Alternatively, separate repos for UI and server may be used if desired.
- **Coding Standards:** Agree on style rules (e.g. Airbnb JS style) and enforce via linters. Use TypeScript for type safety and easy IDE support.
- **Documentation:** Provide a CONTRIBUTING guide, code of conduct, and documentation for setting up the dev environment. Use typed code comments and possibly automated docs (e.g. TypeDoc for TS).
- **Version Control:** Git flow with feature branches. Code review on all PRs. Include template issues/PRs.

16. Performance Diagram (Mermaid)

For clarity, below is a Mermaid diagram of the **system architecture and data flow** described above:

```

flowchart TB
    subgraph Sources
        A["Crypto Exchanges (WS)"]:::ws
        B["Stock/FX APIs (REST)"]
    end
    subgraph Ingestion
        C["Ingestion Service"]:::service
        D["Normalizer"]:::service
        E["Redis Cache"]:::db
        F["Redis/Kafka PubSub"]:::db
    end
    subgraph Backend
        G["WebSocket Server"]:::server
    end
  
```

```

    H[Time-Series DB]:::db
end
subgraph Client
    I[Desktop App (UI)]:::ui
end

A --> C
B --> C
C --> D
D --> E
D --> F
E --> H
F --> G
H --> G
G --> I

classDef ws fill:#f9f,stroke:#333,stroke-width:1px;
classDef service fill:#efe,stroke:#333,stroke-width:1px;
classDef server fill:#eef,stroke:#333,stroke-width:1px;
classDef db fill:#ffe,stroke:#333,stroke-width:1px;
classDef ui fill:#eef,stroke:#333,stroke-width:1px;

```

(Green=services, yellow=databases, purple=websockets, blue=UI)

17. Minimal Viable Architecture and Cost Estimates

A recommended **minimum architecture** (for a basic viable product) is:

- **Frontend:** Electron/Tauri app with a single chart component and minimal UI (React + a chart library).
- **Backend:** One service instance that ingests from one crypto exchange and one equity API, publishes via WebSocket. In-memory cache (Redis) for last prices.
- **Database:** Lightweight TSDB instance (QuestDB or SQLite for MVP).
- **Deployment:** Docker containers for services (optional for Windows-only MVP).

Effort and Cost Estimates (no paid data)

Component	Implementation Details	Team/Role	Rough Effort (person-days)	Notes on Cost
App Framework & UI	Set up Electron/Tauri, React, chart canvas scaffolding; basic glass UI theme.	2× Frontend Dev	~30	~\$15k (dev time)
Chart Engine	Integrate chosen open chart lib (e.g. Lightweight Charts); implement candlestick, zoom, pan.	1× Frontend Dev	~20	~\$10k

Component	Implementation Details	Team/Role	Rough Effort (person-days)	Notes on Cost
Data Ingestion	Write service to fetch crypto WS & one stock API; normalize messages.	1× Backend Dev	~15	~\$7.5k
WebSocket Server	Implement WS server to relay normalized data to client; manage subscriptions.	1× Backend Dev	~10	~\$5k
Database	Integrate TSDB (e.g. QuestDB) for history; write ingest hook.	1× Backend Dev	~10	~\$5k
Indicator Engine	Basic framework for adding one or two indicators (e.g. SMA, RSI).	1× Frontend Dev	~10	~\$5k
UI/UX Enhancements	Polish UI, add CSS glassmorphism, animations.	1× UI/UX Designer & Frontend Dev	~15	~\$7.5k
Testing & CI	Set up unit tests, end-to-end test for chart updates; CI pipelines.	1× DevOps/Backend	~10	~\$5k
Packaging/Installer	Configure build to produce Windows .exe; code signing.	1× DevOps	~5	~\$2.5k
Project Management	Architecture docs, project planning, coordination.	1× Project Manager	Ongoing (~20 days)	~\$10k

Total (approx): ~135 person-days (~8 person-months). At a blended rate (e.g. \$1000/day including overhead), this is on the order of **\$135k** of development effort. (Costs vary by region and dev rates; no paid data costs included.)

This MVP would support one crypto and one equity feed, one multi-series chart, and basic interactivity. Additional markets, more indicators, and advanced performance work would add time.

18. Project Roadmap (Sample)

The project can be divided into sprints (~2 weeks each). Below is a sample plan:

Sprint	Dates	Goals / Deliverables	Tasks	Team Roles
Sprint 1	July 1–14, 2026	Define architecture & tech selection. - Finalize lib/frameworks - Create design docs/diagrams.	Compare libraries (charts, DB, frameworks). Set up repo and basic CI.	PM, Architect, Dev team

Sprint	Dates	Goals / Deliverables	Tasks	Team Roles
Sprint 2	July 15–28, 2026	Prototype UI shell. - Electron/Tauri boilerplate - Simple chart placeholder - Basic UI layout (menus, controls).	Scaffold desktop app; Integrate chosen frontend framework; Test "Hello Chart".	Frontend Dev, UI/UX
Sprint 3	July 29–Aug 11, 2026	Implement core chart. - Integrate charting library with real data. - Enable zoom/pan, axes.	Embed Chart.js/ECharts/Lightweight; Draw static sample data; Implement glass UI elements.	Frontend Dev
Sprint 4	Aug 12–25, 2026	Data pipeline MVP. - Connect to one crypto WS (e.g. Binance). - Display live updates on chart.	Write ingestion service; Normalize and push via WS; Client subscribes to feed.	Backend Dev, Frontend Dev
Sprint 5	Aug 26–Sept 8, 2026	Add equity data & indicators. - Fetch one stock via Alpha Vantage. - Implement a moving average indicator.	REST pull stock price; Compute SMA on backend or client; Overlay on chart.	Backend Dev, Frontend Dev
Sprint 6	Sept 9–22, 2026	Refinement & Persistence. - Store historical data to DB. - Optimize performance (Web Workers, etc).	Integrate QuestDB; Load history on chart start; Benchmark rendering speed; Refactor for FPS.	DevOps, Backend, Frontend
Sprint 7	Sept 23–Oct 6, 2026	Testing & Polish. - Unit and E2E tests. - UI/UX polish (animations, menus).	Write automated tests; Fix bugs; Add animation for transitions; Ensure 120 fps on sample data.	QA Engineer, UI/UX, Devs
Sprint 8	Oct 7–20, 2026	Packaging & Launch. - Build installer. - Set up update server. - Beta release.	Configure electron-builder or Tauri bundler; Sign binary; Document installation; Collect beta feedback.	DevOps, PM, Devs

Each sprint has clear deliverables (e.g. “working live chart” in Sprint 4). Roles:

- **Frontend Dev:** Implements UI, charts, animations.
- **Backend Dev:** Builds data ingestion, normalization, and WebSocket services.
- **UI/UX Designer:** Crafts visual themes (glassmorphism) and user flow.
- **DevOps/Engineer:** Sets up CI/CD, builds, deployments.
- **QA/Tester:** Develops and runs tests.
- **Project Manager:** Oversees timeline, documentation, release management.

This roadmap is illustrative. It assumes a small cross-functional team (approx. 2 frontend + 2 backend + 1 QA + 1 PM). Milestones align with alpha (after Sprint 4), beta (Sprint 7), and initial release (post Sprint 8).

Sources: Official docs and reputable blogs were used to evaluate libraries, data APIs, and frameworks. Citations include research on rendering performance [61†L314-L322] , TradingView's own chart library specs [4†L275-L283] [4†L333-L341] , high-performance chart engines [5†L329-L337] [5†L359-L362] , open-source DB comparisons [55†L69-L74] [58†L242-L250] , and modern app framework analyses [35†L87-L96] [37†L146-L154] [42†L1-L4] . These guide the recommendations above.
